

Л.р. №3. Создание базы данных и запросов к ней с помощью средств графовой СУБД neo4j

Тема:

Изучение средств создания базы данных и выполнения запросов к ней с использованием графовой СУБД neo4j.

Цель:

Получить навыки создания базы данных и выполнения запросов к ней с помощью средств графовой СУБД neo4j.

Задание:

- 1) Сформировать базу данных по выбранной предметной области.
- 2) Составить список из 10 запросов к базе данных.
- 3) С помощью шаблонов запросов получить выборку для каждого запроса из п.2).
- 4) В отчёте отразить в графической или текстовой форме содержимое базы данных, шаблоны запросов и полученные выборки с комментариями.

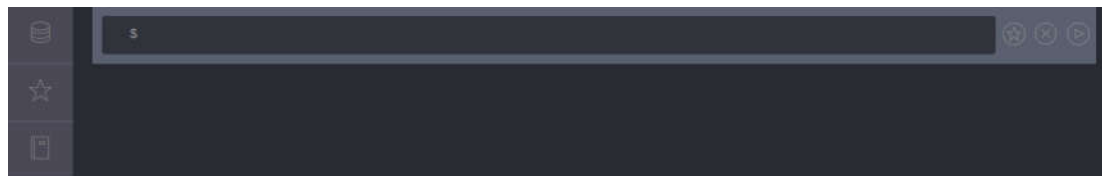
Установка:

- 1) Скопировать сценарий с локального сервера `\\it.lan\Resources\Методические материалы \~Учебные курсы\3 курс\ПБЗ\2016ч1\ЛР3\install_neo4j.sh` на свой компьютер.
- 2) Открыть терминал (Ctrl + Alt + T).
- 3) Перейти в папку со сценарием (cd путь_к_сценарию).
- 4) Запустить сценарий (`./install_neo4j.sh`).
- 5) На все вопросы системы давать положительный ответ.
- 6) Выполнить команду `sudo neo4j start`.
- 7) По умолчанию СУБД доступна по адресу `//localhost:7474`.

Пункт 6) необходимо выполнять при каждом запуске компьютера.

При первом запуске СУБД попросит авторизоваться (стандартные логин и пароль neo4j), далее будет предложено сменить стандартный пароль на собственный.

Для работы с СУБД neo4j разработан специальный язык запросов **Cypher**. Все запросы помещаются в **Editor**, панель, расположенную в верхней части страницы.



Клавиша **Escape** позволяет разворачивать **Editor** и сворачивать его обратно при повторном нажатии.

Результаты запросов отображаются в специальных окошках ниже. Заголовок окна результата содержит текст запроса на языке **Cypher**. История всех запросов накапливается в нижней части страницы (по принципу стека). При необходимости нужное окно результата можно закрепить в верхней части страницы с помощью кнопки «**Pin at top**» в верхней части окна результата.

Чтобы очистить историю запросов (очистить экран), необходимо выполнить команду **:clear**.

Чтобы получить справку о какой-либо команде, необходимо выполнить **:help command_name**.

Для того чтобы выполнить какую-либо команду, можно воспользоваться кнопкой «**Play**» в правой части **Editor** или клавишей **Enter** (если команда состоит из нескольких строк запуск производится комбинацией клавиш **Ctrl + Enter** (добавление новой строки **Shift + Enter**)).

Основные операторы языка Cypher

Оператор создания новой вершины

CREATE (ee: Person {name: "Emil", from: "Sweden"}), где:

- **()** - ограничивают каждую создаваемую вершину,
- **{}** - в таких скобках записываются свойства создаваемой вершины (свойства представляют собой пары «ключ-значение», предназначенные для описания различных характеристик создаваемой сущности (вершины или связи), например, имя, возраст, продолжительность знакомства),
- **ee** - переменная, указывающая на новую вершину базы данных,
- **Person** - метка для новой вершины базы данных, используемая для группировки вершин во множества; каждая вершина может иметь одну и более меток или не иметь их вовсе, однако в операторе **CREATE** можно задать не более одной метки,
- свойство **name** имеет значение "Emil", свойство **from** - "Sweden".

Примечание: строковые литералы можно задавать как в двойных (""), так и в одинарных (') кавычках, т. е. записи

{name: "Emil"} и {name: 'Emil'}

будут восприняты системой совершенно одинаково.

Cypher поддерживает создание нескольких вершин одним оператором **CREATE**, в этом случае каждая новая вершина заключается в круглые скобки **()**, которые разделяются запятой, например:

```
CREATE (ee: Person {name: "Emil", from: "Sweden"}), (bb: Person {name: "Bill", from: "USA"})
```

Оператор **CREATE** также позволяет создавать связки отношений между вершинами:

- Можно *создавать вершины и связи между ними одновременно*:

```
CREATE (ee: Person {name: "Emil", from: "Sweden"})  
-[:KNOWS {how_long: 7}]-> (bb: Person {name: "Bill", from: "USA"})
```

- Или *создавать связь между заранее созданными вершинами*:

```
CREATE (ee: Person {name: "Emil", from: "Sweden"}), (bb: Person {name: "Bill", from: "USA"})  
CREATE (ee) -[:KNOWS {how_long: 7}]-> (bb)
```

Поддерживается создание только ориентированных (**-[...]->**, **<-[...]-**) связей.

Границей связи, в отличие от вершины, являются **[]** скобки.

Связи, как и вершины, могут иметь свойства, которые аналогичным образом перечисляются в **{}** скобках.

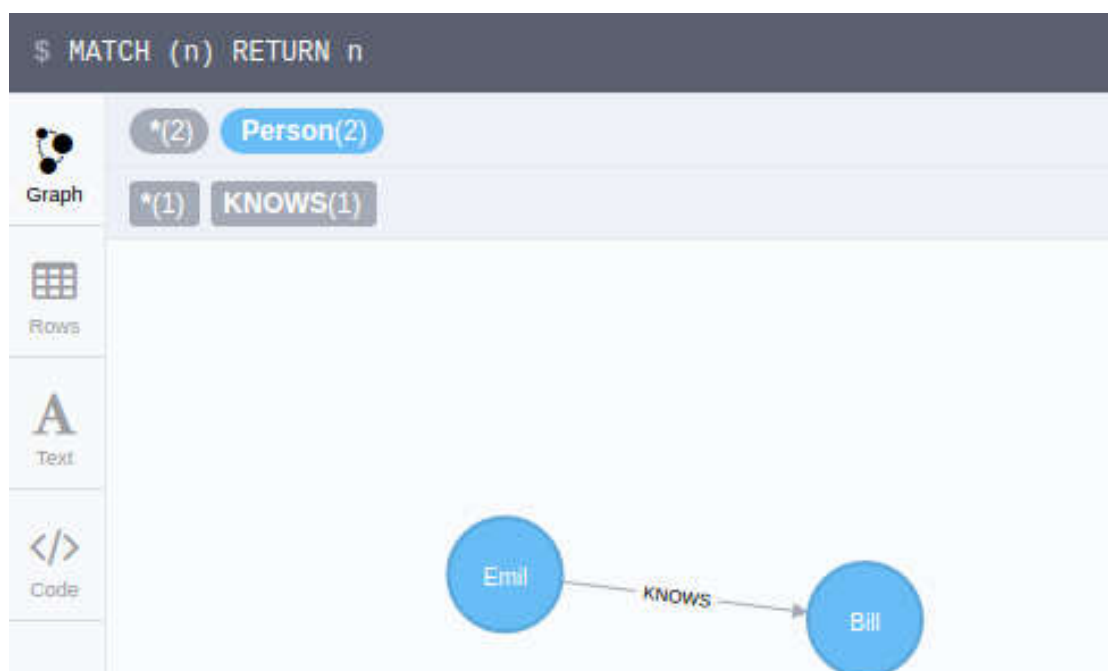
Перед названием типа отношения обязательно ставится двоеточие.

Для того, чтобы *найти* определённые *вершины* в БД, используется оператор **MATCH ... [WHERE ...] RETURN ...** (операторы **MATCH** и **RETURN** являются обязательными, а **WHERE** - опциональным)

Для того, чтобы просмотреть *все вершины, которые есть в БД* на данный момент, необходимо выполнить команду

```
MATCH (n) RETURN n
```

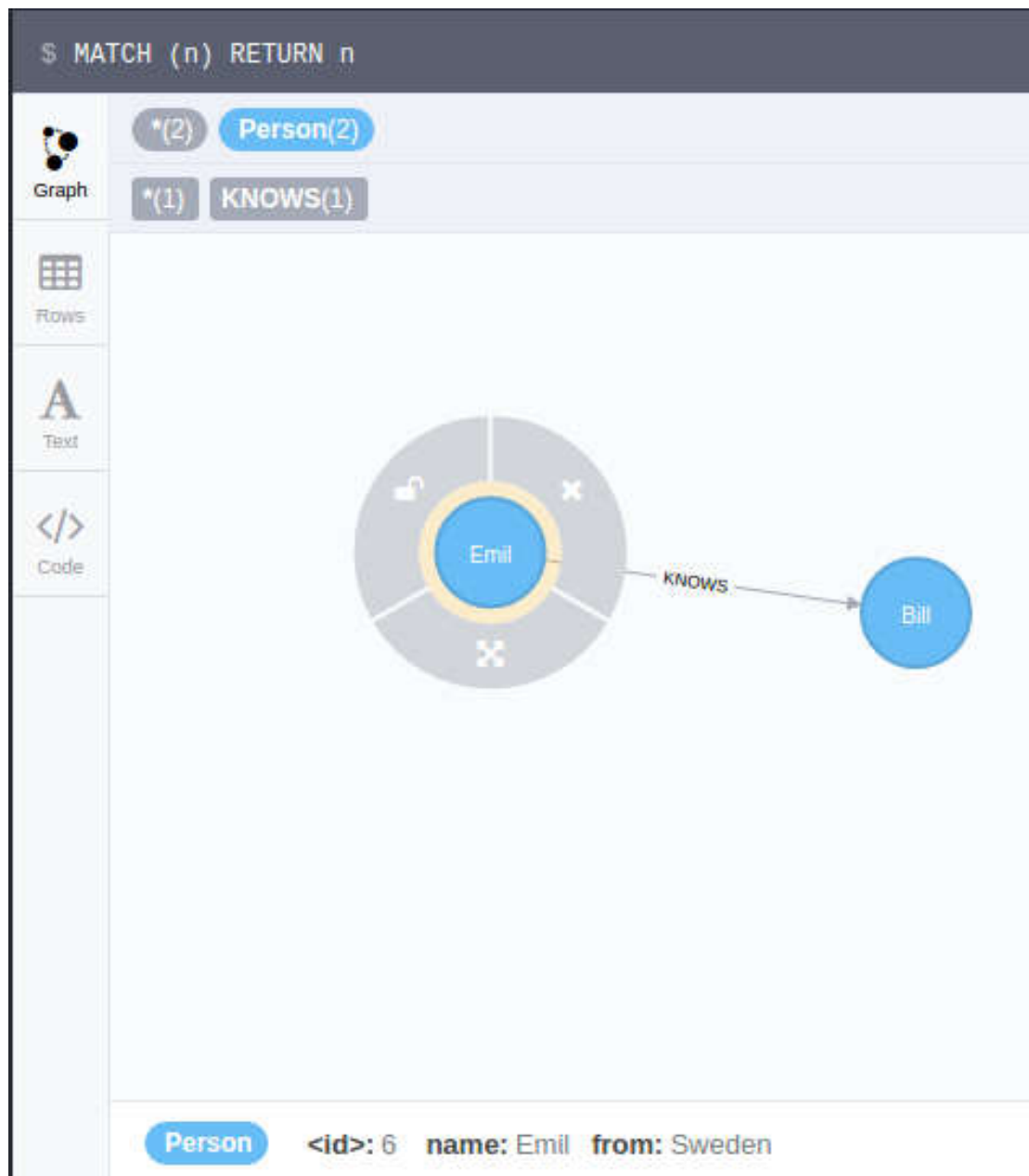
Для созданного выше примера результат будет следующим:



В верхней части результирующего окна отображается количество найденных узлов (с разделением на группы по меткам) и связей отношений (с разделением по типу отношения).

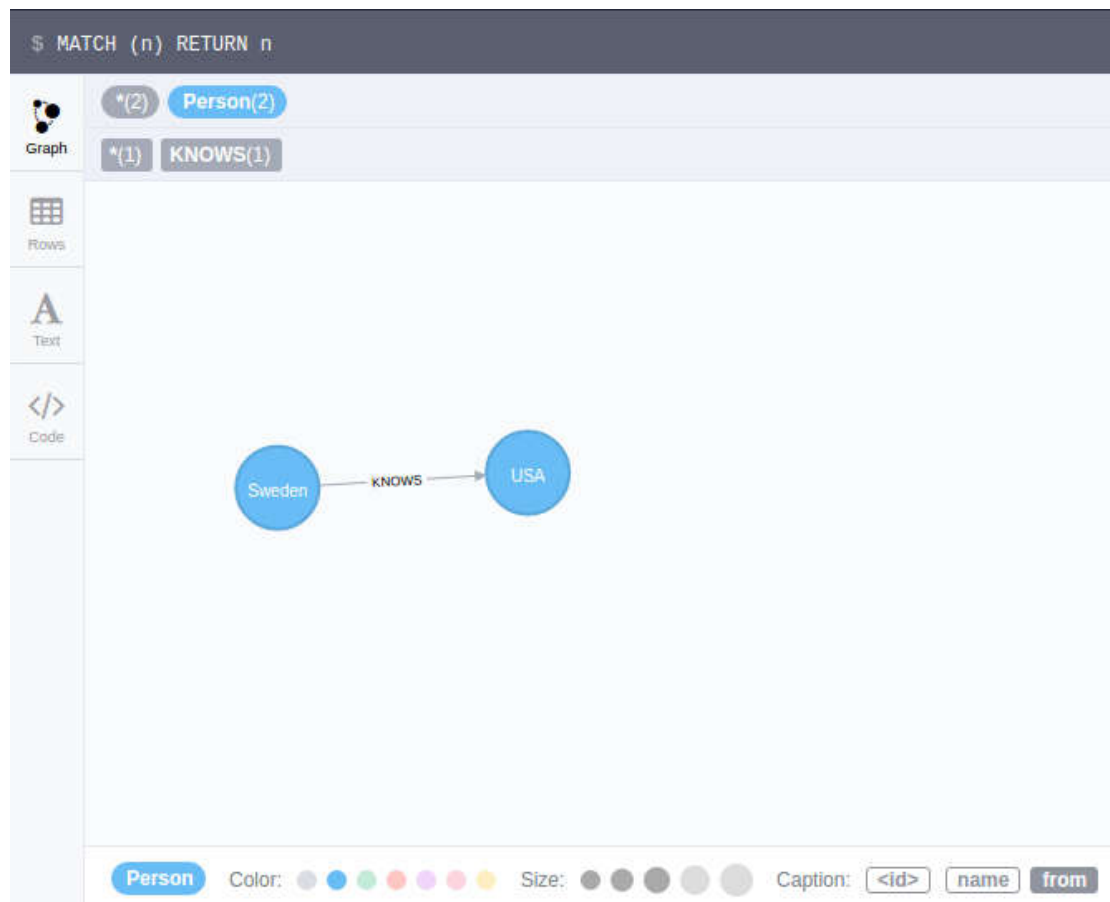
С помощью кнопок, расположенных в левой части результирующего окна, можно менять представление данных (граф, таблица, текст, исходный код).

Для того, чтобы просмотреть все свойства найденного узла, необходимо нажать на узел, и в нижней части окна будут отображены значения всех свойств рассматриваемой вершины:



Данное меню позволяет также настраивать отображение самих вершин: удалить вершину (только с рисунка), разблокировать для переконфигурации графа, развернуть связи (т. е. отобразить все входящие и выходящие дуги вершины, для этого также можно просто дважды нажать на саму вершину).

Для того, чтобы выбрать, какое из свойств вершины или отношения будет являться идентификатором каждого элемента соответствующей группы, необходимо в верхней части результирующего окна выбрать нужную метку или отношение, затем в нижней части окна выбрать желаемое свойство:



Так же можно выбрать цвет и размер элементов группы.

Для того, чтобы *найти* все *вершины с определённым значением какого-либо атрибута*, например, всех людей с именем Emil, необходимо после оператора **WHERE** указать значение известного свойства (свойств). После оператора **WHERE** может также быть указано какое-либо логическое выражение (простое или составное (для составных выражений используются логические операторы **AND**, **OR** и **NOT**)), например:

... **WHERE** n.age <> 0 ... или ... **WHERE** n.age > 16 **AND** n.age < 30 ...

Для рассматриваемого примера запрос к БД будет иметь следующий вид:

```
MATCH (n: Person) WHERE n.name="Emil" RETURN n
```

без оператора **WHERE** запрос можно переписать следующим образом:

```
MATCH (n: Person {name: "Emil"}) RETURN n
```

В обоих случаях результат будет иметь следующий вид:



Если *ничего не известно об узлах, но известно отношение* между ними, тогда запрос к БД будет иметь следующий вид:

```
MATCH (n) - [:KNOWS] - (m) RETURN n,
```

Оператор **MATCH ... RETURN** поддерживает неориентированные связи (`-[...]-`) в качестве шаблона поиска.

Если известна *информация* только *об одном из узлов* и *об отношении* между узлами (например, необходимо найти всех людей, которых знает Emil), тогда можно сделать следующий запрос к БД:

```
MATCH (n: Person {name: "Emil"}) - [:KNOWS] -> (m) RETURN n, m
```

Если при поиске какой-либо промежуточный элемент *не представляет интереса*, можно оставить в шаблоне пустые скобки: `()` - для вершины, `[]` - для связи.

Если искомые вершины могут быть соединены *отношением любого типа*, то в шаблоне поиска такое отношение обозначается `-[*]-`.

Если необходимо *найти* какое-то *определённое количество узлов*, после оператора **RETURN** необходимо использовать оператор **LIMIT amount**, где **amount** - количество необходимых вершин.

Так же можно отсортировать найденные данные с помощью оператора **ORDER BY** (**ASC** - прямой порядок сортировки, **DESC** - обратный).

Если необходимо вывести *в качестве результата* не всю информацию, а только *некоторые свойства*, тогда после оператора **RETURN** необходимо указать, какое именно свойство нужно вывести. Например:

```
MATCH (n: Person {name: "Emil"}) -[:KNOWS] -> (m) RETURN m.name
```

Результат будет иметь следующий вид:

```
$ MATCH (n:Person {name:"Emil"}) -[:KNOWS] -> (m) RETURN m.name
```

	m.name
Rows	Bill
	
Text	
	
Code	

При таком варианте запроса графовое представление результата недоступно.

Для того, чтобы *удалить элемент из БД* используется оператор **MATCH ... [WHERE ...] DELETE ...**

Примечание: в neo4j невозможно удалить вершину, если она связана с другими какими-либо отношениями.

Сначала нужно *удалить связи* этой вершины с другими:

```
MATCH (n: Person) OPTIONAL MATCH (n) -[r] - (m) DELETE r
```

Оператор **OPTIONAL MATCH** означает, что, если какая-либо пара вершин не связана искомым отношением, но удовлетворяет остальным требованиям, она будет отмечена.

Затем удалить *вершины*:

```
MATCH (n: Person) DELETE n
```

Или *удалить всё сразу*:

```
MATCH (n: Person) OPTIONAL MATCH (n) -[r] - (m) DELETE n, r, m
```

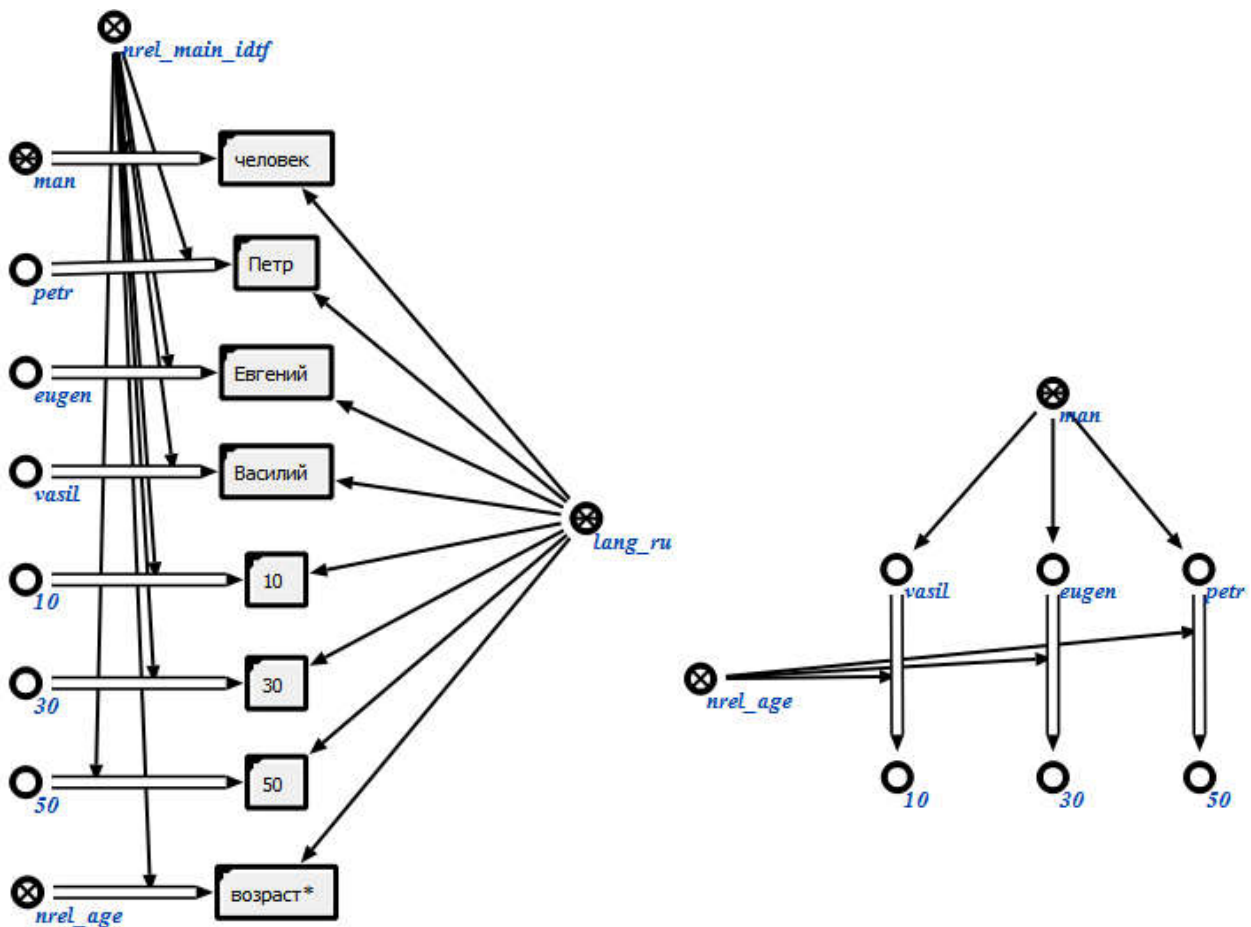
или

MATCH (n: Person) - [r] - (m) **DELETE** n, r, m (так как гарантировано, что все вершины в примере связаны отношением).

Существует и *другой способ задания атрибутов* некоторой вершины: задание значений атрибутов *не с помощью свойств вершины* графовой БД, а *с помощью явного указания* соответствующих *отношений*. Такой подход является более близким к теории семантических сетей, поэтому именно его рекомендуется использовать для выполнения данной лабораторной работы.

Рассмотрим пример:

Пусть *имеется фрагмент БЗ* на языке SCg:

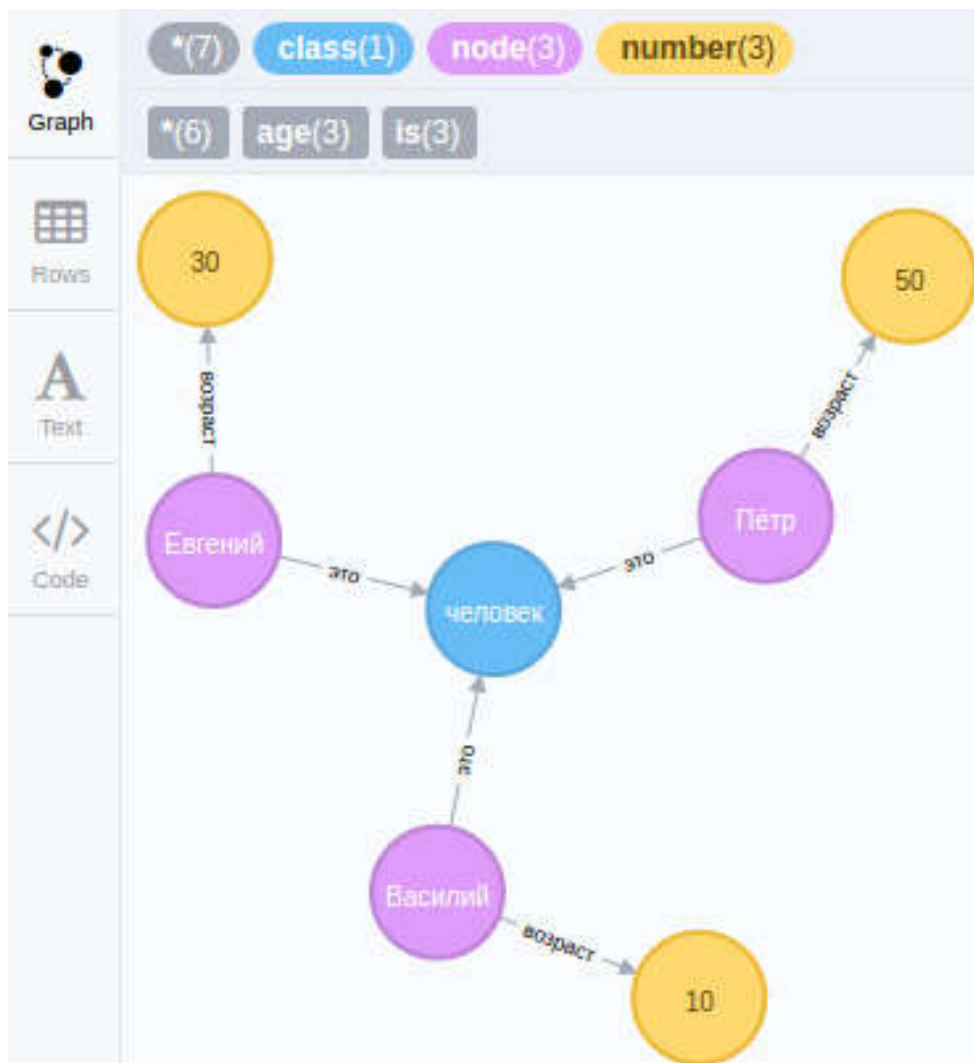


Создадим его эквивалент в neo4j:

```
CREATE (man: class {name: "человек"}),
(ten: number {value: 10}),
(thirty: number {value: 30}),
(fifty: number {value: 50}),
(fifty) <-[:age {name: "возраст"}]-(petr: node {name:
"Пётр"})-[:is {name: "это"}]->(man),
(thirty) <-[:age {name: "возраст"}]-(eugen: node {name:
"Евгений"})-[:is {name: "это"}]->(man),
```

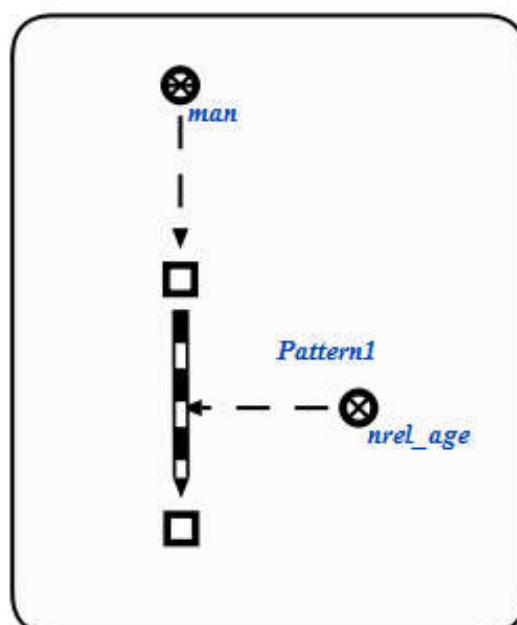


```
(ten)<-[:age {name: "возраст"}]-(vasil: node {name:
"Василий"})-[:is {name: "это"}]->(man)
```



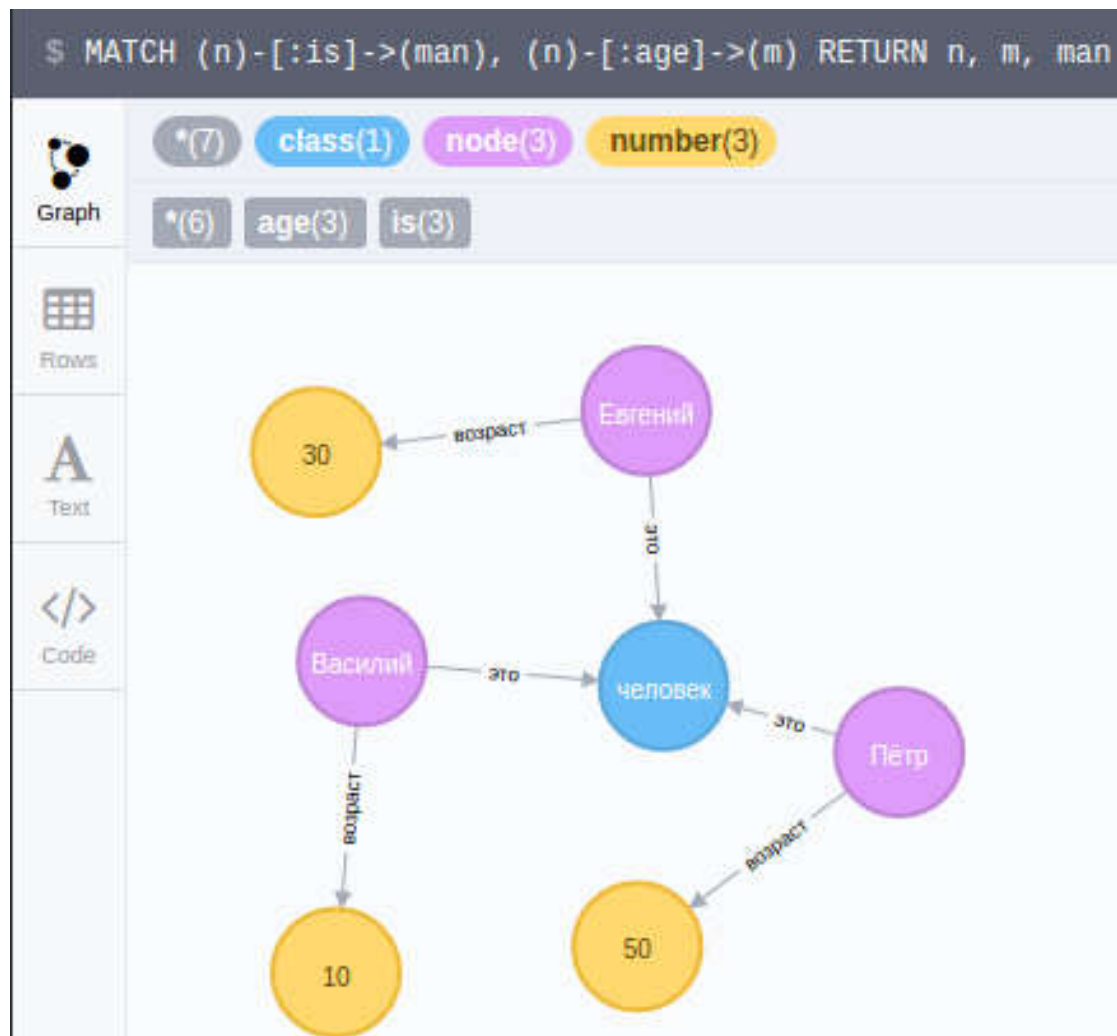
Найдём возраст каждого человека, известного системе:

На SCg шаблон поиска выглядит следующим образом:

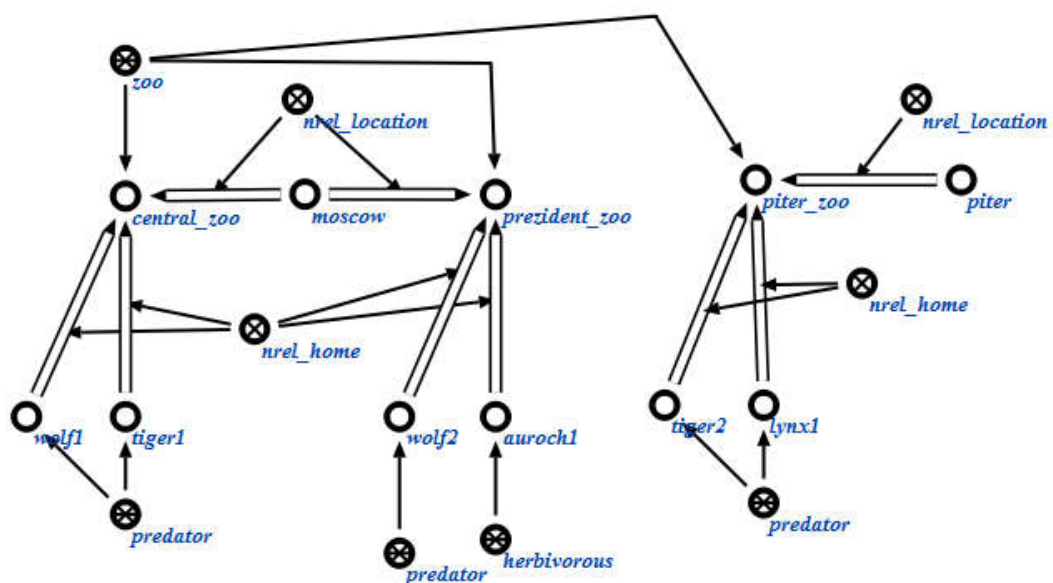


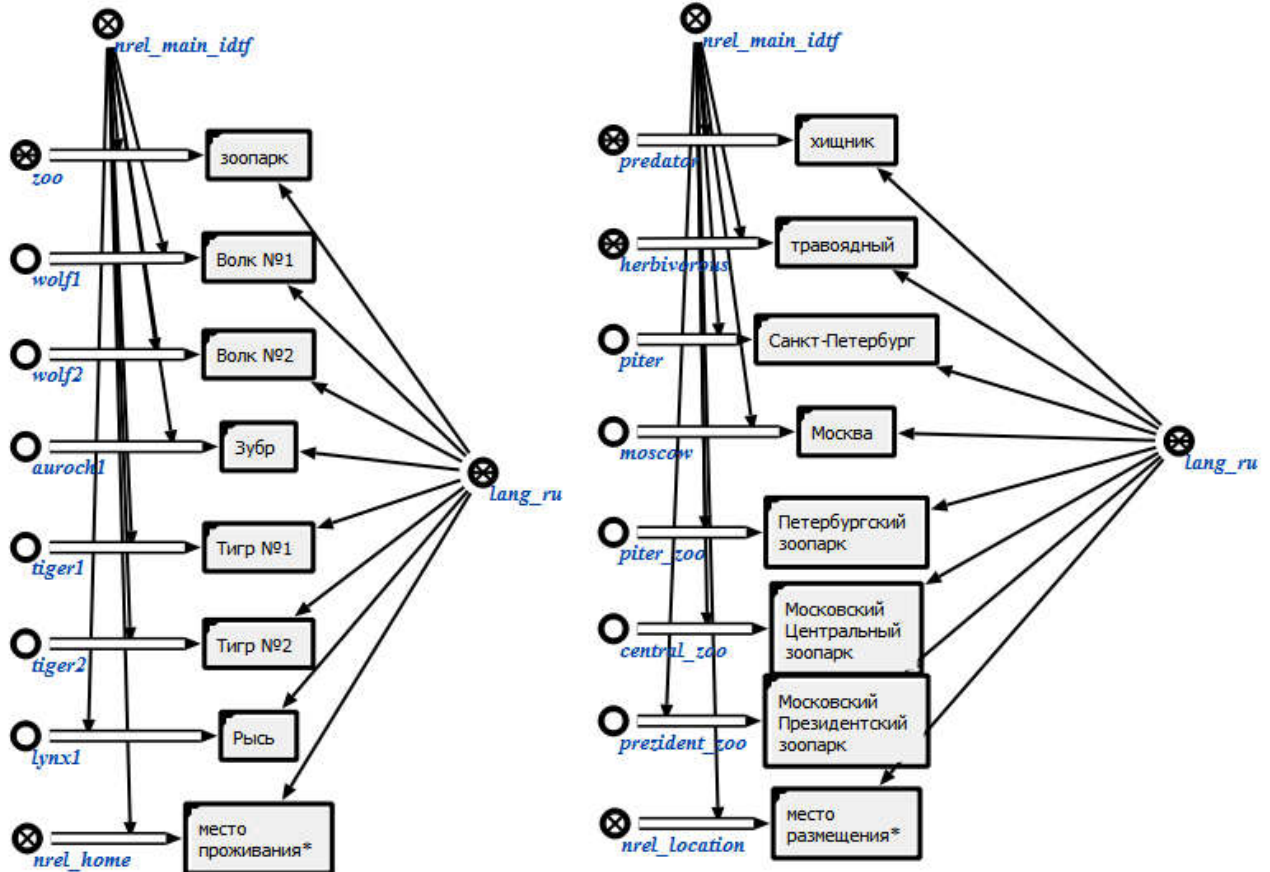
Создадим *аналогичный запрос к БД* в neo4j:

```
MATCH (n)-[:is]->(man), (n)-[:age]->(m) RETURN n, m, man
```



Рассмотрим *другой фрагмент* на языке SCg:





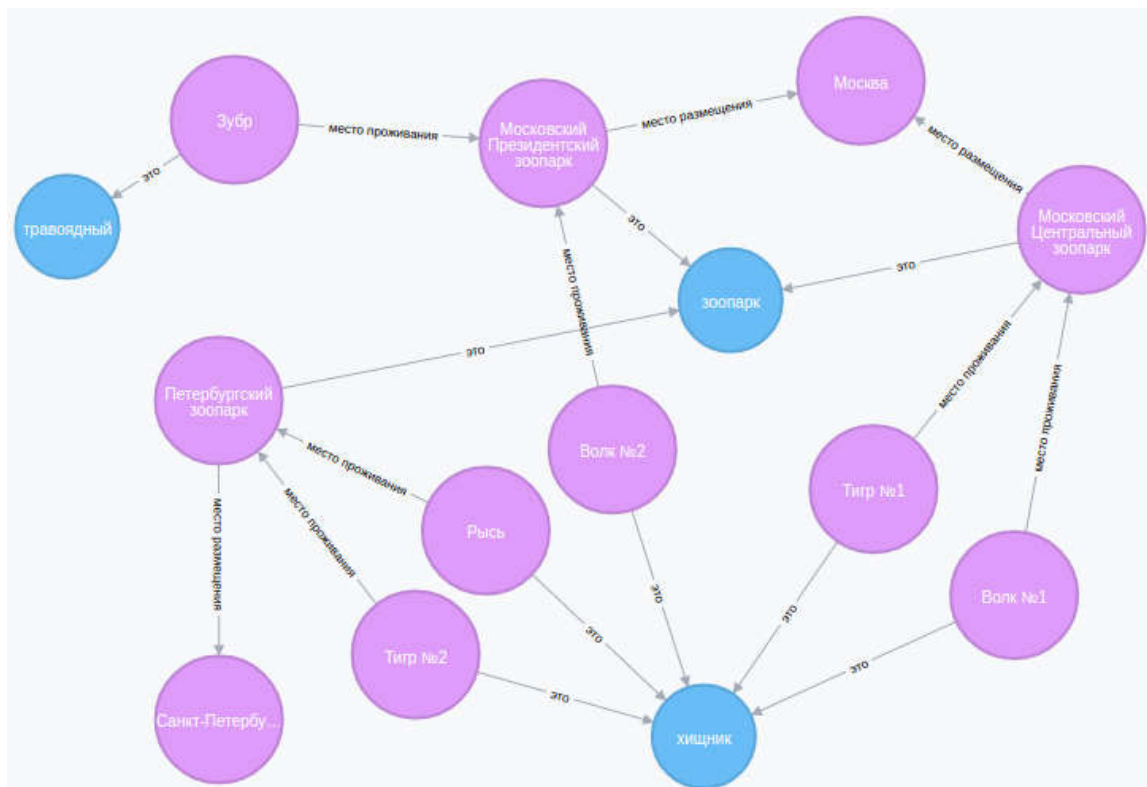
В neo4j он будет выглядеть следующим образом:

```
CREATE (zoo: class {name: "зоопарк"}),
(wolf1: node {name: "Волк №1"}),
(wolf2: node {name: "Волк №2"}),
(auroch: node {name: "Зубр"}),
(tiger1: node {name: "Тигр №1"}),
(tiger2: node {name: "Тигр №2"}),
(lynx: node {name: "РЫСЬ"}),
(predator: class {name: "хищник"}),
(herbivorous: class {name: "травоядный"}),
(piter: node {name: "Санкт-Петербург"}),
(moscow: node {name: "Москва"}),
(piter_zoo: node {name: "Петербургский зоопарк"}),
(central_zoo: node {name: "Московский Центральный зоопарк"}),
(prezident_zoo: node {name: "Московский Президентский зоопарк"}),
(zoo) <-[:is {name: "это"}]-(central_zoo)-[:location {name: "место размещения"}]->(moscow),
(predator) <-[:is {name: "это"}]-(wolf1)-[:home {name: "место проживания"}]->(central_zoo),
```

```

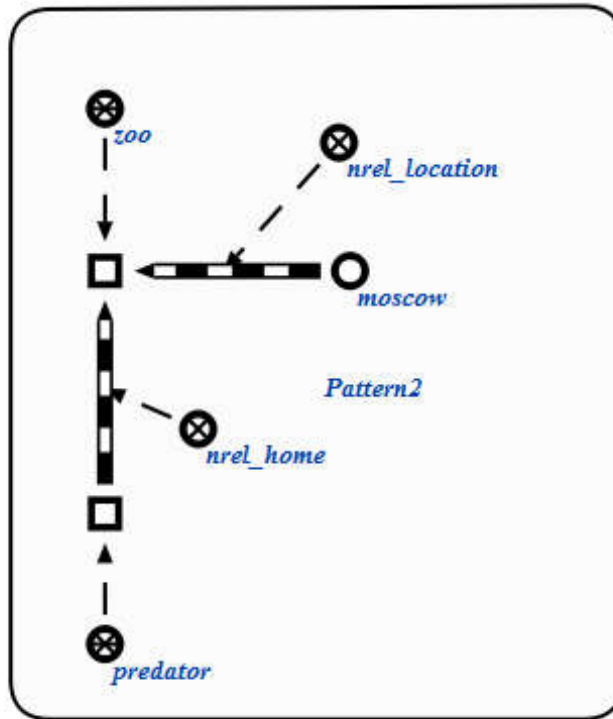
(predator)<-[:is {name: "это"}]-(tiger1)-[:home {name: "место
проживания"}]->(central_zoo),
(zoo)<-[:is {name: "это"}]-(prezident_zoo)-[:location {name:
"место размещения"}]->(moscow),
(predator)<-[:is {name: "это"}]-(wolf2)-[:home {name: "место
проживания"}]->(prezident_zoo),
(herbivorous)<-[:is {name: "это"}]-(auroch)-[:home {name:
"место проживания"}]->(prezident_zoo),
(zoo)<-[:is {name: "это"}]-(piter_zoo)-[:location {name:
"место размещения"}]->(piter),
(predator)<-[:is {name: "это"}]-(tiger2)-[:home {name: "место
проживания"}]->(piter_zoo),
(predator)<-[:is {name: "это"}]-(lynx)-[:home {name: "место
проживания"}]->(piter_zoo)

```



Найдём всех **хищников**, находящихся **в зоопарках в Москве**:

Шаблон поиска на языке **SCg** имеет следующий вид:



Соответствующий запрос к БД на языке Cypher будет иметь вид:

```
MATCH (moscow {name: "Москва"})<-[:location]-(n)-[:is]->(zoo
{name: "зоопарк"}),
(predator {name: "хищник"})<-[:is]-(p)-[:home]->(n) RETURN
moscow, n, zoo, predator, p
```

